

Secure Shift

Onboarding for the frontend



1.Welcome

Welcome to the SecureShift Frontend Panel Team!

Congratulations on joining the project. We are excited to have you on board and look forward to working with you throughout this trimester.

SecureShift is a large, real-world software engineering project where students collaborate in agile teams to develop a workforce management platform for the security industry. As members of the Frontend Team, you will contribute to building intuitive, reliable, and scalable user interfaces while working closely with the Backend and Mobile teams.

Whether you are new to React or already have frontend development experience, this guide will help you understand the project, set up your development environment, and begin contributing effectively.

2.About SecureShift

SecureShift is a workforce management platform designed to simplify the daily operations of security companies.

The system enables employers to manage every aspect of their workforce from a single application.

Main Features

- Shift Management
- Employee Management
- Attendance Tracking
- Payroll Management
- Incident Reporting
- Equipment Management
- Employer and Guard Communication
- Notifications
- Multi-language Support
- Dashboard and Analytics

Example Workflow

Without SecureShift:

Employer creates shifts manually → Contacts guards individually → Attendance is recorded on paper → Payroll is calculated manually.

With SecureShift:

Employer creates a shift → Guard receives notification → Guard accepts shift → Guard checks in using the mobile app → Attendance is recorded automatically → Payroll is generated digitally.

3. Project Architecture

The project consists of three major components:

Frontend Panel

The Frontend Panel is developed using React and serves as the web interface for the SecureShift platform.

Currently, the frontend consists of the Employer Panel, which allows employers to manage:

- Guards
- Shifts
- Payroll
- Timesheets
- Incident Reports
- Equipment
- Notifications
- Daily Activity Tracking
- Reports

This trimester, one of our primary objectives is to develop the new Admin Panel. The Admin Panel will provide administrators with a dedicated interface to manage the overall system, including users, roles, platform settings, monitoring, and other administrative functions. The frontend team will be responsible for designing, implementing, and integrating this new panel with the existing backend APIs.

Current Focus: Employer Panel (Existing)

Trimester 2 Goal: Design and develop the new Admin Panel while continuing to maintain and enhance the Employer Panel.

This wording clearly explains the current state of the project and highlights what the new frontend team will be working on.

Backend

Developed using Node.js and Express.

Provides REST APIs, authentication, business logic, and database communication.

Guard Mobile App

Developed using React Native.

Allows guards to apply for shifts, check in/out, report incidents, receive notifications, and communicate with employers.

All three systems communicate through the backend APIs.

4. Technology Stack

Technology	Purpose
React	Frontend Development
React Router	Navigation
Axios	API Communication
Material UI	User Interface Components
Node.js	Backend Development
Express	REST API Framework
MongoDB	Database
Docker	Local Development Environment
GitHub	Version Control
MS Planner	Sprint Management
Swagger	API Documentation
Postman	API Testing

5. Employer Panel Overview

The Employer Panel allows employers to manage the entire workforce through an easy-to-use web interface.

Some of the pages include:

- Dashboard
- Shift Management
- Timesheets
- Payroll
- Guard Management
- Incident Reports
- Equipment Management
- Notifications
- Chat
- Daily Activity Tracking
- Profile Management

Most frontend tasks involve creating user interfaces, integrating APIs, handling application state, validating forms, and improving user experience.

6. Development Environment Setup

Before writing any code, complete the following steps.

Step 1

Clone both repositories.

- Frontend Repository
- Backend Repository

Step 2

Install:

- Node.js
- Docker Desktop

- Visual Studio Code
- Git

Step 3

Run the project using Docker.

Verify that:

- MongoDB starts successfully
- Backend runs without errors
- Frontend loads correctly
- Login page is accessible

Important: Many issues experienced by previous teams were caused by incomplete Docker setup. Ensure your environment is working before beginning development.

Video Link- How to get project in to your locals- [2026-07-12 10-47-12.mkv](#)

Part2-

https://1drv.ms/v/c/4916f816167acfd5/IQATx7k7xjEGRJSCV19Wpr2EARx_9Kbl5IcberniYdFI?e=9BaD0U

1. Clone the repository

Open PowerShell:

```
cd "C:\Users\.....\SIT378_SIT782 - Team Project B - Execution and Delivery"
```

Clone:

```
git clone https://github.com/Gopher-Industries/SecureShift.git
```

Go into the project:

```
cd SecureShift
```

2. Create your own branch

Check branches:

```
git branch
```

Update main:

git checkout main

git pull origin main

Create your branch:

git checkout -b frontend/.....

Confirm:

git branch

You should see:

* frontend/.....

main

Backend Setup

3. Go to backend folder

cd app-backend

Check files:

dir

4. Create .env

Create a file:

app-backend/.env

Add:

MONGO_URI=

PORT=

JWT_SECRET=

SMTP_HOST=

SMTP_PORT=

SMTP_SECURE=

Save.

Docker Setup

5. Go back to project root

`Cd ..`

You should be here:

`SecureShift`

Check:

`dir`

You should see:

`docker-compose.yml`

`app-backend`

`app-frontend`

6. Check Docker is installed

`docker --version`

`docker compose version`

7. Stop old containers (if already running)

Because you already have containers:

`docker compose down`

8. Build and start containers

Run:

`docker compose up --build -d`

Wait until it completes.

9. Check containers

`docker ps`

Expected:

securshift-backend

securshift-db

securshift-frontend-employer

Testing

10. Backend

Open:

<http://localhost:5000>

If you see:

Cannot GET /

that is OK.

11. Swagger API

Open:

<http://localhost:5000/api-docs>

You should see API documentation.

12. Frontend

Open:

<http://localhost:3000>

You should see:

Employer Panel

Useful Docker Commands

See logs

Backend:

`docker logs secureshift-backend`

Frontend:

`docker logs secureshift-frontend-employer`

Database:

`docker logs secureshift-db`

Restart everything

`docker compose restart`

Stop everything

`docker compose down`

After making code changes

`docker compose up --build -d`

Git workflow after changes

Check changes:

`git status`

Add:

`git add .`

Commit:

`git commit -m "Setup local environment"`

Push your branch:

`git push origin frontend/.....`

7. Understanding the Frontend Project Structure

You will mainly work within the following folders:

src/

components/

pages/

layouts/

hooks/

services/

routes/

assets/

utils/

Example

Suppose you are assigned the task:

"Improve the Payroll Page."

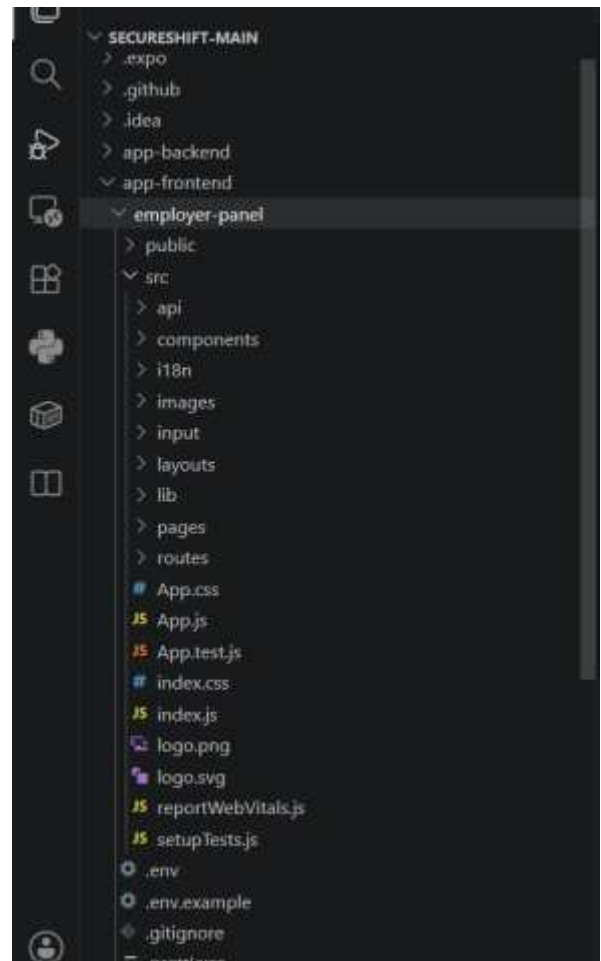
You may need to modify:

pages/Payroll.jsx

services/payrollService.js

components/PayrollTable.jsx

The page displays the data, the service communicates with the backend, and the component presents reusable UI elements.



8. How Frontend Communicates with Backend

Most pages follow the same flow:

1. User performs an action.
2. React component captures the event.
3. Axios sends a request to the backend API.
4. Backend processes the request.

5. MongoDB stores or retrieves data.
6. Backend returns a response.
7. React updates the interface.

Example

When an employer approves a shift:

User clicks "Approve"

↓

React calls

POST /api/shifts/approve

↓

Backend validates request

↓

Database updates

↓

Frontend refreshes

↓

Status changes to "Approved"

9.Git Workflow

We follow a feature-branch workflow.

Never develop directly on the **main** branch.

Typical workflow:

1. Pull the latest main branch.
2. Create a new feature branch.
3. Implement your feature.
4. Test locally.

5. Commit changes.
6. Push your branch.
7. Create a Pull Request.
8. Wait for review before merging.

Example branch names:

frontend/yourname-payroll-page

frontend / yourname -dashboard-improvements

frontend / yourname -chat-notification

frontend / yourname -equipment-management

10.Coding Standards

To maintain consistency across the project:

- Follow ESLint rules.
- Use Prettier formatting.
- Write reusable React components.
- Keep components modular.
- Avoid unnecessary duplication.
- Write meaningful commit messages.
- Keep Pull Requests focused on one feature.

11.Current Project Status

The previous team completed significant development across the project.

Employer Panel

Completed:

- Timesheets
- Payroll
- Daily Activity Tracking

- Employer Dashboard
- Chat System
- Notifications
- Equipment Management
- Multi-language Support
- Privacy Policy
- Terms and Conditions
- UI Improvements
- Bug Fixes
- Security Improvements

12.Current Priorities for This Trimester

As the new Frontend Team, our priorities include:

- Completing remaining backend integrations.
- Implementing newly designed Figma pages.
- Improving application performance.
- Fixing existing bugs.
- Strengthening frontend security.
- Increasing test coverage.
- Supporting AI feature integration.
- Improving overall user experience.

13.Known Issues

Some issues remain from the previous trimester.

Guard Page

Currently blocked due to a backend Role-Based Access Control (RBAC) issue.

The frontend implementation is mostly complete but cannot retrieve guard data until the backend issue is resolved.

14.Team Communication

Our primary communication platforms are:

- Microsoft Teams
- GitHub
- MS Planner

Please keep your Planner tasks updated and communicate any blockers early.

If you are stuck on a task for an extended period, ask for assistance rather than waiting.

15.Pull Request Checklist

Before submitting a Pull Request, ensure that:

- The application builds successfully.
- Your feature has been tested locally.
- ESLint passes without errors.
- No unnecessary files are included.
- Screenshots are attached (where applicable).
- The Pull Request description clearly explains your changes.

16.Resources

Please familiarise yourself with the following resources before starting development:

- GitHub Repository
- Swagger API Documentation
- System Architecture Document
- Entity Relationship Diagram (ERD)
- Figma Designs
- User Manual

- MS Planner

17. Tips for Success

- Pull the latest changes regularly.
- Test before committing.
- Keep commits small and meaningful.
- Read the API documentation before integrating endpoints.
- Ask questions whenever something is unclear.
- Collaborate with teammates and support one another.
- Focus on writing clean, maintainable code rather than rushing to complete tasks.

18. Final Message

SecureShift is more than a university assignment—it is an opportunity to experience professional software development practices in a collaborative environment.

Throughout this trimester, you will work with modern technologies, follow Agile methodologies, participate in code reviews, collaborate across multiple teams, and contribute to a real-world application.

Every contribution, whether it is a new feature, a bug fix, or a code improvement, plays an important role in the success of the project.

On behalf of the Frontend Team, welcome to SecureShift. We look forward to working with you and wish you a successful and enjoyable trimester.